CYNBIT TECHNOLOGIES

Backend Technical Audit Report & Architecture Review

Passive Observational Assessment, Security Evaluation, and Decoupled FastAPI Reference Backend

Executive Overview & Audit Context

This formal technical document presents an external, non-intrusive backend technical audit of Cynbit Technologies' public digital web ecosystem (https://cynbit.in/ and portal.cynbit.in). Conducted as part of the Backend Developer Technical Assessment, this report evaluates API architecture, authentication posture, data validation patterns, edge caching behavior, and transport security in accordance with the OWASP Web Security Testing Guide (WSTG v4.2).

The report includes a 10-point technical findings register, an 18-point classified evidence register, remediation roadmaps, and an architectural blueprint for a standalone production-grade reference backend built with Python 3.11, FastAPI, MySQL 8.0, SQLAlchemy 2.x, Alembic, and JWT.

AUDIT SPECIFICATION & METADATA

Candidate / Lead Auditor:	Chahat Modi
Academic Institution:	JECRC University, Jaipur (Final-Year B.Tech CSE)
Role Applied For:	Entry-Level Backend Developer / Software Engineer
Target Organization:	Cynbit Technologies (Jaipur, Rajasthan, India)
Target Web Applications:	https://cynbit.in/ (Canonical: www.cynbit.in) & portal.cynbit.in
Hosting & Framework:	Vercel Global Edge (Mumbai BOM1) | Next.js App Router (RSC)
Audit Methodology:	OWASP WSTG v4.2 Passive Observational & Transport Reconnaissance
Assessment Date:	November 1, 2026
Submission Status:	Final Verified Version 1.0

1. Executive Summary

A passive, non-intrusive backend technical audit of Cynbit Technologies' public digital infrastructure was conducted on November 1, 2026. The target surfaces comprise the primary website (https://cynbit.in/ and its canonical endpoint https://www.cynbit.in/) and the dedicated internal portal (https://portal.cynbit.in/).

Key Engineering Highlights:

1. **Modern Edge Infrastructure: Cynbit utilizes the Vercel Global Edge Network with Mumbai (BOM1) termination,** achieving optimal regional latency for domestic visitors. Pre-rendered HTML assets exhibit high cache hit ratios.
2. **Modern Full-Stack Architecture: Next.js App Router powers the web application, leveraging React Server** Components (RSC) and Next.js Server Actions for state mutations.
3. **Cryptographic Transport Security: The platform enforces TLS 1.3 with AES-128-GCM cipher negotiation and an** automated Let's Encrypt certificate. HTTP Strict Transport Security (HSTS) is configured with a 2-year preload.
4. **Authentication Risk on Portal: The internal administrative portal utilizes a single-factor, shared 'Security Key'** password field without individual user accounts (no username/email) or multi-factor authentication (MFA).
5. **Missing Security Headers: The edge lacks defense-in-depth headers including Content-Security-Policy (CSP),** X-Frame-Options (Clickjacking defense), and X-Content-Type-Options: nosniff.
6. **Mutation Decoupling Opportunity: Contact inquiries and candidate applications rely on coupled Server Actions** rather than an independently documented, testable RESTful API with published OpenAPI/Swagger contracts.

2. Assessment Scope and Methodology

The technical audit adhered strictly to passive, authorized reconnaissance standards defined in the OWASP Web Security Testing Guide (WSTG v4.2). The scope encompassed five observational dimensions:

· Network & DNS Layer: Authoritative queries for A, MX, TXT (SPF), NS, and DMARC records.
· Transport & Cryptographic Layer: Direct TLS 1.3 socket negotiation, certificate chains, and HTTP security headers.
· Application & Framework Layer: Next.js App Router routing patterns, SSG edge caching, and robots.txt paths.
· Data Handling Layer: Client-side JavaScript bundle reverse engineering for /contact, /careers, and /portal.
· Authentication & Session Layer: Verification of administrative credential flows and localStorage persistence.

EVIDENCE CLASSIFICATION FRAMEWORK

[OBSERVED]: Directly verified through HTTP response headers, script code, certificate exchange, or DNS responses.

[INFERRED]: A logical, technical deduction based on visible architectural patterns and framework behaviors.

[POTENTIAL RISK]: An engineering risk identified from observable configurations requiring internal review to confirm.

[NOT VERIFIABLE]: An internal backend capability (e.g. database query parameters, indexing, server-side queues).

3. Website Architecture · Publicly Observable View

Through passive network telemetry, the external hosting and edge topology was mapped:

Component	Network Target	Observed Technical Architecture
Apex Domain	https://cynbit.in/	Resolves to Vercel IP 216.198.79.1. Emits HTTP 307 to www.
Canonical URL	https://www.cynbit.in/	Primary public site; Next.js App Router on Vercel.
Internal Portal	https://portal.cynbit.in/	Dedicated portal subdomain; also redirected from /portal.
Edge Infrastructure	Vercel BOM1 (Mumbai)	Terminates TLS 1.3; serves cached SSG assets.
Email Infrastructure	Microsoft 365 Exchange	MX record: cynbit-in.mail.protection.outlook.com.

4. API Behaviour and Structure (Next.js Server Actions)

Standard API discovery indicated that traditional REST endpoints (/api, /api/v1) return HTTP 404 pages.
Instead, Cynbit utilizes Next.js Server Actions executed via client-server RPC references:

Server Action Name	Cryptographic Action Hash	Page Origin
`submitInquiry`	`40247ba4465344b3386595bab4303317cdcce907df`	/contact
`submitCandidateApplication`	`40645a00e04f6854903cf3a65fda4c75ddd7691de9`	/careers/[slug]
`loginToPortal`	`40d93da1cc26de4ef37bcb273184b7141ef77a50f8`	portal.cynbit.in/

Technical Assessment:

While Server Actions eliminate boilerplate for internal React mutations, they introduce tight coupling between frontend build artifacts and backend logic. They lack standardized OpenAPI/Swagger documentation, which restricts external integration, automated API fuzzing, and client reusability. In our reference build, we demonstrate how decoupling mutations into a standalone FastAPI REST API establishes clear contracts.

5. Authentication & Administrative Portal Analysis

De-obfuscation of the portal bundle (app/portal/page-ba479285e84c64b5.js) revealed the login routine:

```
let l = (0, s.createServerReference)("40d93da1cc26de4ef37bcb273184b7141ef77a50f8", s.callServer, ...);
async function c(r) {
  r.preventDefault(); let t = new FormData(); t.append("password", e);
  let a = await l(t);
  if (a.success) {
    try { localStorage.setItem("cynbit_portal_last_activity", Date.now().toString()); } catch (e) {}
    window.location.href = "/dashboard";
  } else { s(a.error || "Login failed"); }
}
```

Critical Authentication Findings:

Shared Single-Secret Risk (FIND-001): The portal accepts solely a 'Security Key' with no username/email. All admins share one secret, eliminat

Absence of Multi-Factor Authentication: No secondary authentication (TOTP/WebAuthn) is required for administrative entry.

Client-Side Storage Vulnerability (FIND-002): Session activity is written to localStorage, which is accessible to any script executing within t

Operational Route Guard (EVD-014): Direct unauthenticated GET requests to /dashboard return an HTTP 307 redirect to /, verifying server-side ro

6. Security Headers & CORS Analysis

Security Header / Policy	Observed State	Risk Severity	Impact & Recommendation
Strict-Transport-Security	max-age=63072000	**Compliant**	Enforces HTTPS preload for 2 years.
Content-Security-Policy	Missing	**Medium**	No script/connect source restrictions.
X-Frame-Options	Missing	**Medium**	Susceptible to Clickjacking via iframes.
X-Content-Type-Options	Missing	**Low**	Allows browser MIME-sniffing.
Access-Control-Allow-Origin	Wildcard (*)	**Medium**	Permissive CORS on HTML documents.
X-Powered-By	Next.js Exposed	**Low**	Reveals framework technology family.
DMARC Record	p=none (Monitor)	**Low**	Does not reject or quarantine spoofed mail.

7. Summary of Technical Findings

The technical audit identified 10 distinct findings across authentication, security, and architecture:

Finding ID	Category	Severity	Finding Title & Scope
FIND-001	Auth & Access	**High**	Single-Factor Shared 'Security Key' Administrative Authentication
FIND-002	Session Mgmt	**Medium**	Client-Side Session Activity Storage in LocalStorage
FIND-003	Web Security	**Medium**	Permissive Wildcard Cross-Origin Resource Sharing on HTML Responses
FIND-004	App Security	**Medium**	Absence of Content Security Policy (CSP) Defense Header
FIND-005	App Security	**Medium**	Absence of Clickjacking Defense (X-Frame-Options / frame-ancestors)
FIND-006	App Security	**Low**	Missing X-Content-Type-Options: nosniff Header
FIND-007	Info Disclosure	**Low**	Framework Fingerprinting via X-Powered-By: Next.js
FIND-008	Info Disclosure	**Informational**	Administrative Route Enumeration via robots.txt
FIND-009	Domain Security	**Low**	Inconsistent SPF Records and Unenforced DMARC Policy (p=none)
FIND-010	Architecture	**Informational**	Coupled Server Action Architecture Lacking Decoupled REST API Contract

8. Strategic Remediation Roadmap

Phase 1: Immediate Edge & DNS Hardening (Week 1)

· Update SPF to include spf.protection.outlook.com.
· Elevate DMARC to p=quarantine.
· Disable poweredByHeader in next.config.js.
· Convert apex redirect to HTTP 301 Permanent.

Phase 2: Security Header & CORS Enforcement (Week 2)

· Implement Content-Security-Policy (CSP) with script nonces.
· Enforce X-Frame-Options: DENY and X-Content-Type-Options: nosniff.
· Restrict Access-Control-Allow-Origin strictly to trusted API origins.

Phase 3: Identity & Access Management Overhaul (Weeks 3-4)

· Replace single security key with individual user accounts (email + password).
· Hash passwords using bcrypt (work factor >= 12).
· Issue signed JWT access/refresh tokens with RBAC (Admin, Staff, Auditor).

Phase 4: Decoupled REST API Microservice (Weeks 5-6)

· Extract inquiry and candidate workflows into a dedicated FastAPI REST API.
· Enforce Pydantic schema validation on all inputs.
· Persist records in MySQL 8.0+ managed via Alembic database migrations.

9. Standalone Reference Backend Architecture

To provide Cynbit with a production-ready model addressing these audit findings, a standalone Secure Inquiry Management API was architected using Python 3.11 and FastAPI:

```
cynbit-backend-assessment/
··· app/
·   ··· main.py                    # FastAPI application factory & security middleware
·   ··· core/
·   ·   ··· config.py              # Pydantic BaseSettings environment configuration
·   ·   ··· security.py            # bcrypt password hashing & PyJWT token management
·   ·   ··· exceptions.py          # Global RFC 7807 structured exception handlers
·   ··· database/
·   ·   ··· connection.py          # SQLAlchemy session engine & lifecycle
·   ·   ··· base.py                # Declarative Base for models
·   ··· models/
·   ·   ··· inquiry.py             # Inquiry SQLAlchemy relational model
·   ·   ··· admin.py               # AdminUser relational model with roles
·   ··· schemas/
·   ·   ··· inquiry.py             # Pydantic models for validation & responses
·   ·   ··· auth.py                # Pydantic models for Login, Tokens & Users
·   ··· routers/
·   ·   ··· inquiry.py             # Public inquiry creation & Protected admin CRUD
·   ·   ··· auth.py                # Login, profile, and token refresh endpoints
·   ··· services/
·   ·   ··· inquiry_service.py     # Business logic & repository transactions
·   ·   ··· auth_service.py        # Authentication verification & token issuance
·   ··· dependencies/
·       ··· auth.py                # OAuth2 Bearer token verification & RBAC guard
··· alembic/                       # Version-controlled schema migrations
··· tests/                         # Automated Pytest + HTTPX test suite
··· docs/                          # Technical audit reports, evidence & API documentation
··· requirements.txt               # Locked application dependencies
··· README.md                      # Comprehensive setup & architecture documentation
```

Key Architectural Safeguards Implemented:

- **Decoupled REST Service:** Independent OpenAPI/Swagger documentation at /docs, testable by automated suites.
- **Server-Side Pydantic Validation:** Sanitizes and enforces bounds on email, strings, lengths, and empty whitespace.
- **Enterprise JWT Authentication:** Bcrypt password hashing (work factor >= 12), token expiration, and RBAC guards.
- **Controlled Relational State:** MySQL 8.0 engine managed via SQLAlchemy 2.x and version-controlled Alembic migrations.
- **Defense-in-Depth Middleware:** Global exception handling, strict CORS whitelisting, and secure response headers.

ETHICAL TESTING DECLARATION

All testing was strictly passive, external, and non-intrusive. No exploit payloads were sent, no credentials were brute-forced, and zero data was submitted to Cynbit systems. Prepared with integrity by Chahat Modi.